

GUIA DE CONSULTA: PROGRAMAÇÃO ORIENTADA A OBJETOS

Aluno: Fabricio Zaneratto Ansanello

1. Fundamentos da O.O

A POO modela o software como um conjunto de objetos que interagem entre si.

- **Classe:** O projeto, molde ou gabarito. Define atributos e métodos.
- **Objeto (Instância):** A concretização da classe na memória (**new**).
- **Variáveis de Instância:** Atributos que definem o estado do objeto. Cada instância possui seus próprios valores.
- **Métodos de Instância:** Definem o comportamento do objeto.
- **Métodos Construtores:** * **Função:** Inicializar o objeto. É chamado obrigatoriamente no **new**.
 - **Sobrecarga (Overload):** Criar mais de um construtor com parâmetros diferentes (ex: um construtor só com nome e outro com nome e CPF).
 - **Diferença:** Não possui tipo de retorno e tem o nome da classe.

Java

```
public class Aluno {  
    String nome; // Variável de instância  
  
    // Sobrecarga de Construtores  
    public Aluno() {}  
    public Aluno(String nome) { this.nome = nome; }  
  
    public void estudar() { System.out.println(nome + " está estudando."); }  
}
```

2. Encapsulamento

Para que serve: Proteção e segurança. Impede que o objeto assuma estados inválidos.

Por que usar: Facilita a manutenção. Se a regra de validação mudar, você altera apenas no método set.

- **Private:** Modificador de acesso que esconde o atributo do mundo exterior.
- **Getters e Setters:** * **Getter:** Retorna o valor (Acesso de leitura).
 - **Setter:** Altera o valor com **validação** (Acesso de escrita seguro).

Java

```
public class Produto {  
    private double preco;  
  
    public void setPreco(double preco) {  
        if (preco >= 0) { this.preco = preco; } // Validação e Proteção  
    }  
    public double getPreco() { return preco; }
```

}

3. Associação Todo-Parte

Descreve o vínculo entre um objeto "Todo" e seus objetos "Parte".

- **Agregação:** Relação de "tem um" fraca. A parte existe sem o todo (Ex: Professor e Departamento).
- **Composição:** Relação de "tem um" forte. A parte é destruída junto com o todo (Ex: ItemPedido e Pedido).
- **Classe de Associação:** Surge em relações de Muitos-para-Muitos (\$N:N\$) para armazenar dados do relacionamento (Ex: A classe `Matricula` guarda a `nota` entre `Aluno` e `Disciplina`).
- **ArrayList:** Coleção dinâmica de objetos.
 - **Uso:** Quando não sabemos a quantidade exata de elementos.
 - **Métodos:** `add()`, `remove()`, `size()`, `get()`.

Java

```
import java.util.ArrayList;

public class Curso {
    private ArrayList<Aluno> alunos = new ArrayList<>(); // Associação Todo-Parte

    public void matricular(Aluno a) { alunos.add(a); }
}
```

4. Diagrama de Classe da UML

Representação visual da estrutura do sistema.

- **Estrutura da Classe:** Retângulo com 3 partes: **Nome** (Topo), **Atributos** (Meio), **Métodos** (Base).
- **Visibilidade:** `+` (public), `-` (private), `#` (protected).
- **Relações Visuais:**
 - **Agregação:** Linha com losango vazio no Todo.
 - **Composição:** Linha com losango preenchido no Todo.
 - **Herança:** Linha com triângulo vazio apontando para a Superclasse.

5. Herança-Associação

Permite que classes compartilhem atributos e métodos através de uma hierarquia "É UM".

- **Superclasse vs. Subclasse:** A superclasse é genérica; a subclasse é especializada.
- **Extends:** Palavra-chave que ativa a herança.

- **@Override:** Informa que você está reescrevendo um método da superclasse para mudar o comportamento (fundamental para garantir a correção do código).
- **Polimorfismo:** Capacidade de uma variável da Superclasse referenciar um objeto da Subclasse e executar o método específico da filha.
- **super():** Chama o construtor da Superclasse.
- **super.metodo():** Chama o método original da Superclasse.

Java

```
public class Animal {
    public void emitirSom() { System.out.println("Som genérico"); }
}

public class Gato extends Animal {
    @Override
    public void emitirSom() { System.out.println("Miau!"); } // Polimorfismo
}
```

6. Classes e Métodos Abstratos

Para que serve: Definir modelos ou rascunhos de classes que não fazem sentido existir sozinhas.

- **Classe Abstrata:** Marcada com `abstract`. Não pode ser instanciada (`new`).
- **Método Abstrato:** Não possui corpo {}. **Obriga** as subclasses a implementarem a lógica.
- **Quando usar:** Quando você tem uma base comum, mas cada filho faz a ação de um jeito (Ex: `Forma` -> `CalcularArea()`).

Java

```
public abstract class Conta {
    public abstract void atualizar(); // Método abstrato
}
```

7. Classes Interfaces

O que é: Um contrato de comportamento. Define o que a classe **deve fazer**.

- **Contrato:** Se uma classe implementa uma interface, ela é obrigada a fornecer código para todos os métodos dela.
- **Uso:** Quando classes sem parentesco precisam da mesma funcionalidade (Ex: `Smartphone` e `Carro` podem implementar `Bluetooth`).
- **Comparação com Abstrata:** Na interface, **todos** os métodos são abstratos e você pode implementar várias. Na abstrata, pode haver métodos prontos e você só herda de uma.

Java

```
public interface Autenticavel {  
    boolean login(String senha);  
}
```

⚡ SEÇÃO: CAI NA PROVA (Resumo e Dicas)

- **Vantagem da Herança:** Reuso de código.
 - **Vantagem da Interface:** Padronização e flexibilidade (Herança múltipla de comportamentos).
 - **Sobrecarga vs. Sobrescrita:** Sobrecarga é na mesma classe (parâmetros diferentes). Sobrescrita é na herança (mudar o que o pai faz).
 - **Polimorfismo:** Só ocorre se houver herança e sobrescrita. Permite que o código seja genérico.
 - **Protected:** Use quando quiser que apenas a "família" (filhas) veja o atributo, mas mantenha-o escondido do resto do sistema.
-

Exemplo Integrado (Empresa)

Java

```
abstract class Funcionario {  
    protected double salario;  
    public abstract double calcularBonus();  
}  
  
class Gerente extends Funcionario implements Autenticavel {  
    @Override  
    public double calcularBonus() { return this.salario * 0.2; }  
  
    @Override  
    public boolean login(String senha) { return senha.equals("123"); }  
}
```